

Theory

**1. What is the time complexity of insertion, deletion, and searching when a BST is skewed (all nodes having only left children or only right children)?**

Skewed means the nodes are unbalanced. So Time Complexity for everything will be  $O(n)$

---

**2. What is the time complexity of searching a node in a balanced BST?**

$O(\log n)$

Each comparison eliminates half the remaining nodes. So search, insert, and delete all run in  $O(\log n)$ . That's why binary search trees are more efficient than binary trees.

---

**4. What does the in-order traversal always produce in a BST?**

An ascending sorted sequence of elements.

Reverse inorder gives descending order

---

**5. What is the degree of a leaf node in a binary tree?**

0 (A leaf node has: No left child & No right child)

---

**6. Suppose that we have numbers between 1 and 100 in a Binary Search Tree and want to search for the number 55. Which of the following sequences CANNOT be the sequence of nodes examined?**

A. (10, 75, 64, 43, 60, 57, 55)

B. (90, 12, 68, 34, 62, 45, 55)

**C.** (9, 85, 47, 68, 43, 57, 55)

**D.** (79, 14, 72, 56, 6, 53, 55)

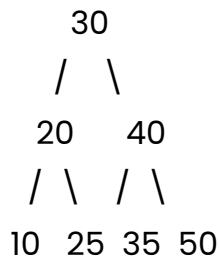
Ans: D. (79, 14, 72, 56, 6, 53, 55)

---

### 7. Preorder to Postorder (in BST)

**Preorder:** 30, 20, 10, 25, 40, 35, 50

Ans:



Post Order: 10,25,20,35,50,40,30

---

### 8. The preorder traversal sequence of a Binary Search Tree (BST) is:

**Preorder:** 30, 20, 10, 15, 25, 23, 39, 35, 42

**Which of the following is the postorder traversal sequence of the same tree?**

**[Pre to Post]**

A. 10, 20, 15, 23, 25, 35, 42, 39, 30

B. 15, 10, 25, 23, 20, 42, 35, 39, 30

C. 15, 20, 10, 23, 25, 42, 35, 39, 30

D. 15, 10, 23, 25, 20, 35, 42, 39, 30

Ans: D

---

**9. Time complexity to check if a tree is a Binary Tree -  $O(n)$**

---

**10. Covert this post order BST to preorder: 1 7 5 50 40 10**

Ans:

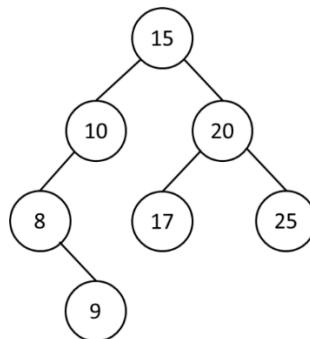
```
    10
   /  \
  5    40
 /  \   \
1   7   50
```

Preorder: 10 5 1 7 40 50

---

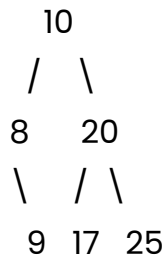
**11.**

Consider the following BST.



- A. Delete the root 15 with its predecessor and draw the resulting tree.
- B. Is the tree drawn on A a complete binary tree (Yes / No)?
- C. Is the tree drawn on A a balanced binary tree (Yes / No)?

A.



B. No. As one right child is missing

C. Yes

$|\text{height}(\text{left subtree}) - \text{height}(\text{right subtree})| \leq 1$

---

**12. The Worst-case Time Complexity for the insert operation of a heap and an unbalanced BST is the same.**

False.

Worst-case for

Heap -  $O(\log n)$

Unbalanced BST -  $O(n)$

---

**13. Can a BST have duplicate keys?**

Ans: Normally, no

Depends on implementation

Standard BST:  No duplicates

Modified BST:  Allowed

---

**14. Every inorder traversal of a binary tree produces a sorted sequence.**

Ans: False

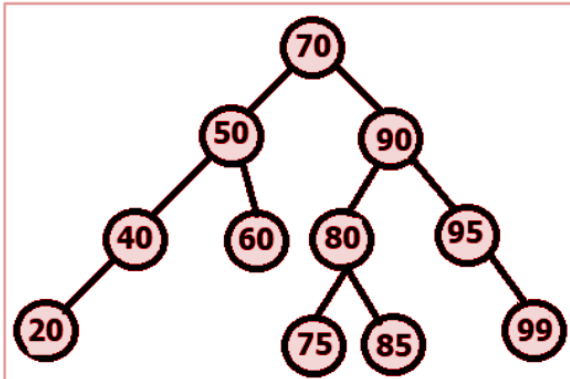
Only Binary Search Trees guarantee sorted inorder traversal.



# Basics

- Codes for Height, Node Count, Inorder/Preorder/Postorder Traversal for BST are done in the same way as they are done in Binary Tree
- **Ascending e Print korte bolle Inorder Print hobe**

## Insert Node



**Insert 10**

**Insert 5**

**Must Make Balanced**

```
public static BTNode addNode(BTNode root, int a) {
```

```
    //Kono root nai
```

```
    if (root == null) {
        BTNode n = new BTNode(a);
        return n;
    }
```

```
    //Root r left, right faka hole direct boshbe
```

```
    if (a < root.elem && root.left == null) {
        BTNode n = new BTNode(a);
        root.left = n;
        return root;
    }
```

```
    if (a > root.elem && root.right == null) {
        BTNode n = new BTNode(a);
```



```

    root.right = n;
    return root;
}

//Root r left right faka NA hole recursive call
if (a < root.elem && root.left != null) {
    addNode(root.left, a);
}
else if (a > root.elem && root.right != null) {
    addNode(root.right, a);
}

return root;
}

```

---

## Searching

```

public static boolean search(BTNode root, int key) {

    if (root == null) {
        return false;
    }

    if (root.elem == key) {
        return true;
    }

    if (key < root.elem) {
        return search(root.left, key);
    }
    else {
        return search(root.right, key);
    }
}

```

---

## Find Minimum value (Maximum is found in a similar way)

```
public static int getMin(BTNode root) {
    while (root.left != null) {
        root = root.left;
    }
    return root.elem;
}
```

---

## Find the Inorder Predecessor

```
public static BTNode getPredecessor(BTNode root, int key) {

    BTNode pred = null;
    BTNode curr = root;

    while (curr != null) {

        if (key == curr.elem) {
            if (curr.left != null) {
                pred = curr.left;           //Will go left
                while (pred.right != null) pred = pred.right; //Continue to go right
            }
            break;
        }

        if (key < curr.elem) {
            curr = curr.left;
        }
    }
}
```

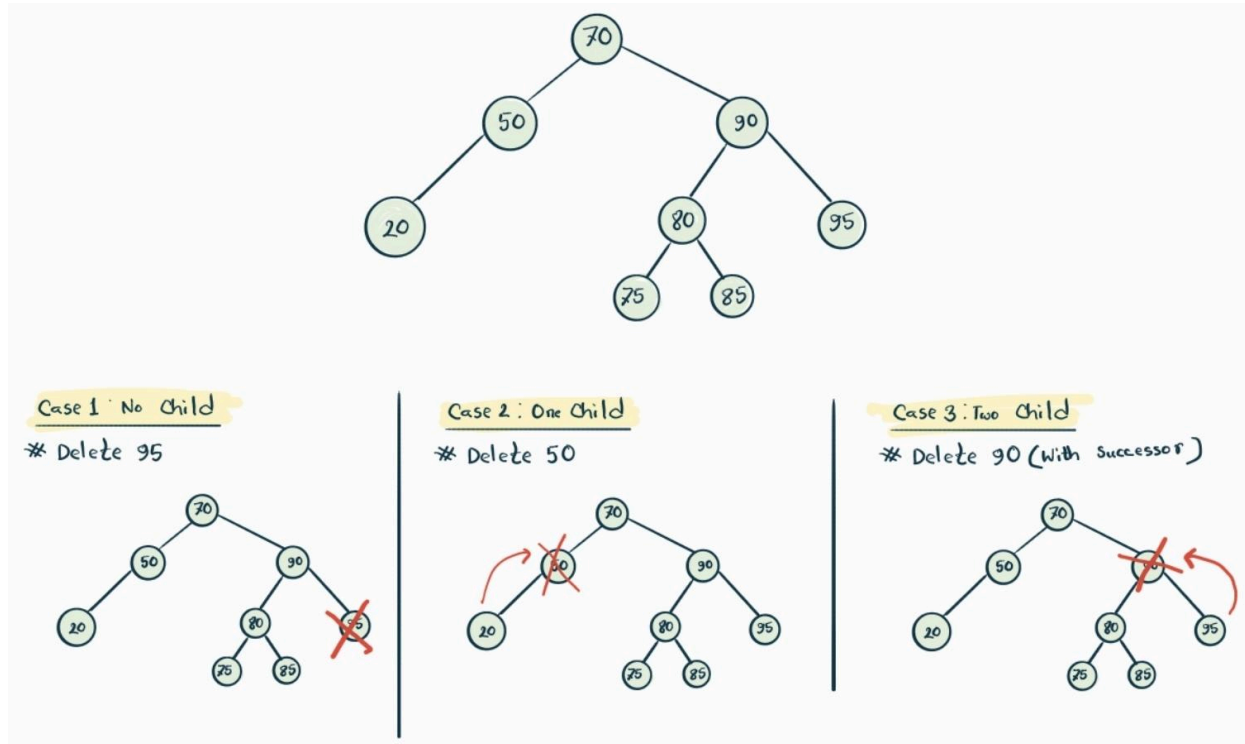
```
        else if (key > curr.elem) {  
            pred = curr;  
            curr = curr.right;  
        }  
    }  
  
    return pred;  
}
```

---

## Find Successor

```
public static BTNode getSuccessor(BTNode root, int key) {  
  
    BTNode succ = null;  
    BTNode curr = root;  
  
    while (curr != null) {  
  
        if (key == curr.elem) {  
            if (curr.right != null) {  
                succ = curr.right;  
                while (succ.left != null) succ = succ.left;  
            }  
            break;  
        }  
  
        if (key < curr.elem) {  
            succ = curr;  
            curr = curr.left;  
        }  
        else curr = curr.right;  
    }  
  
    return succ;  
}
```

## Delete Node



```
public static BTNode deleteNode(BTNode root, int key) {
```

```
    if (root == null) return null;
```

```
    if (key < root.elem){  
        root.left = deleteNode(root.left, key);  
    }
```

```
    else if (key > root.elem){  
        root.right = deleteNode(root.right, key);  
    }
```

```
    else {
```

```

    // CASE 1: No child
    if (root.left == null && root.right == null) return null;

    // CASE 2: One child
    if (root.left == null) return root.right;
    if (root.right == null) return root.left;

    // CASE 3: Two children (using predecessor)
    BTreeNode pred = rightMost(root.left);
    root.elem = pred.elem;
    root.left = deleteNode(root.left, pred.elem);
}

return root;
}

//helper
public static BTreeNode rightMost(BTreeNode n) {
    while (n.right != null) n = n.right;
    return n;
}

public static BTreeNode leftMost(BTreeNode n) {
    while (n.left != null) n = n.left;
    return n;
}

```

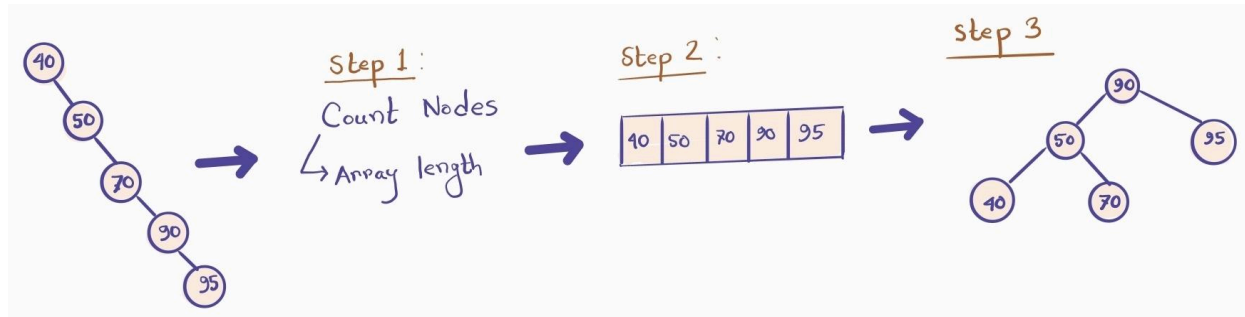
---

```

/* If you want successor instead:
    BTreeNode succ = leftMost(root.right);
    root.elem = succ.elem;
    root.right = deleteNode(root.right, succ.elem); */

```

## Convert to Balanced BST



```
public static BTreeNode balanceBST(BTreeNode root) {  
    int n = countNodes(root);  
    int[] arr = new int[n];  
    inorderFill(root, arr, 0); // Fill array starting from index 0  
    return buildBalancedFromArray(arr, 0, n - 1);  
}
```

//Step 1: Count nodes

```
public static int countNodes(BTreeNode root) {  
    if (root == null) return 0;  
    return countNodes(root.left) + countNodes(root.right) + 1;  
}
```

//Step 2: Inorder Array

```
private static int inorderFill(BTreeNode root, int[] arr, int index) {  
    if (root == null) return index; //Null hole no change to index  
  
    index = inorderFill(root.left, arr, index);  
    arr[index] = root.elem;  
    index++;  
    index = inorderFill(root.right, arr, index);  
  
    return index;  
}
```

//Step 3: Build balanced BST

```
private static BTNode buildBalancedFromArray(int[] arr, int start, int end) {  
    if (start > end) return null;  
  
    int mid = (start + end) / 2;  
    BTNode root = new BTNode(arr[mid]);  
  
    root.left = buildBalancedFromArray(arr, start, mid - 1);  
    root.right = buildBalancedFromArray(arr, mid + 1, end);  
  
    return root;  
}
```

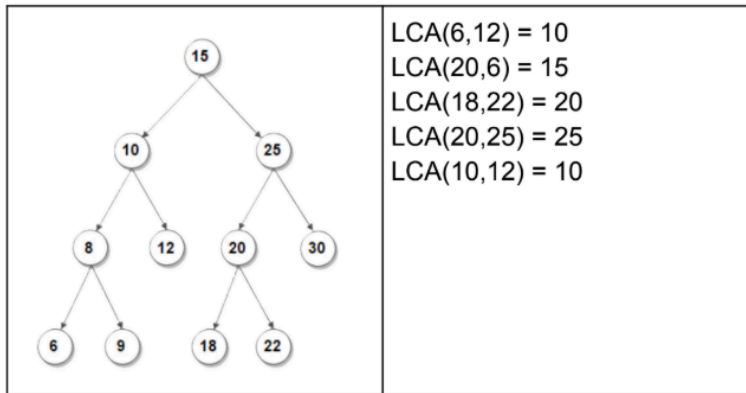
# Lab Task



## 1. Find the Lowest Common Ancestor (LCA) of two nodes in a BST

The lowest common ancestor (LCA) of two nodes  $x$  and  $y$  in the BST is the lowest (i.e., deepest) node that has both  $x$  and  $y$  as descendants. In other words, the LCA of  $x$  and  $y$  is the shared ancestor of  $x$  and  $y$  that is located farthest from the root.

Corner Case: if  $y$  lies in the subtree rooted at node  $x$ , then  $x$  is the LCA; otherwise, if  $x$  lies in the subtree rooted at node  $y$ , then  $y$  is the LCA.



```
public class Task1 {
```

```
    public static Integer lowestCommonAncestor(BSTNode root, Integer x, Integer y) {
```

```
        BSTNode curr = root;
```

```
        while (curr != null) {
```

```
            if (x < curr.elem && y < curr.elem) {
```

```
                curr = curr.left;
```

```
            }
```

```
            else if (x > curr.elem && y > curr.elem) {
```

```
                curr = curr.right;
```

```
            }
```

```
            else {
```

```
                return curr.elem;    // Ekta boro, ekta choto hole found
```

```
            }
```

```
        }
```

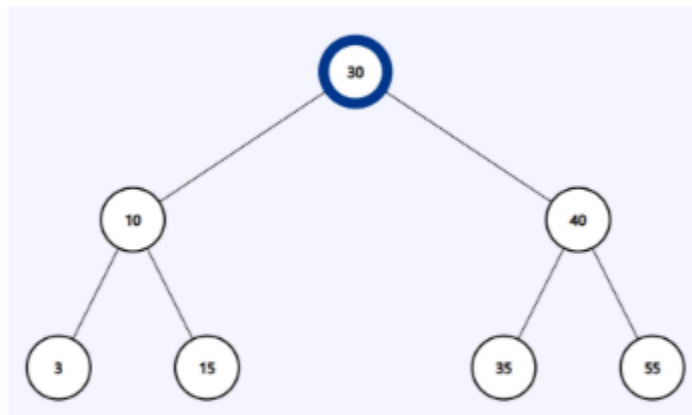
```
        return null; // should not occur if values exist in tree
```

```
}  
}
```

---

A Binary Search Tree is given. The root node is the starting position of Faria and a key node will be given as the destination. Now, you have to find if the following key node (which is the destination of Faria) is present in the tree or not. If present, return the path from the root node to the key node else print "No Path Found".

**[You can use Python List for this task]**



| Sample Input                                       | Sample Output      |
|----------------------------------------------------|--------------------|
| Source node(root): 30<br>Destination node(key): 15 | Path: [30, 10, 15] |
| Source node(root): 30<br>Destination node(key): 50 | No Path Found      |

//Depth 2 hole, can traverse 3 of them

```
public static String findPath( BSTNode root, Integer key ){  
    BSTNode curr=root;  
    String s= "";  
    boolean found= false;
```

```
while(curr!=null){
    if(key==curr.elem){
        found=true;
        break;
    }

    else if(key< curr.elem){
        s+= Integer.toString(curr.elem)+ " ";
        curr=curr.left;
    }

    else if(key> curr.elem){
        s+= Integer.toString(curr.elem)+ " ";
        curr=curr.right;
    }
}

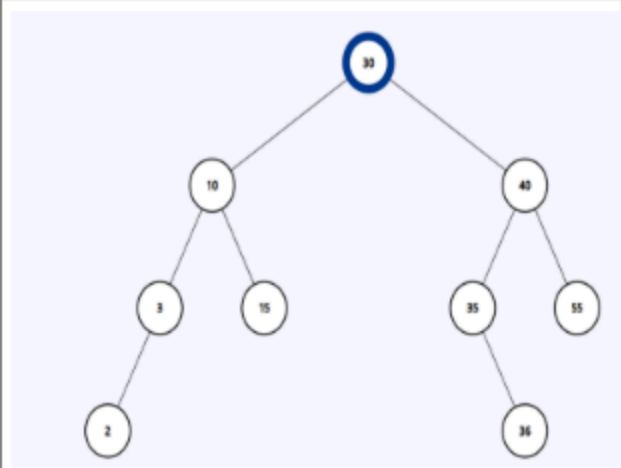
if(found==true){
    s+= Integer.toString(key);
    return s;
}

return "No Path Found";
}
```

---

### 3. Sum of all Leaf nodes of a BST

Write a function **sum\_of\_leaves(root)** that takes the root of a binary search tree as input and prints the sum of all the leaf nodes of the tree.

| Sample Input                                                                      | Sample Output                       |
|-----------------------------------------------------------------------------------|-------------------------------------|
|  | Sum of all<br>leaves=2+15+36+55=108 |

```
public class Task3 {  
  
    public static int sum_of_leaves(BSTNode root) {  
        if (root == null) return 0;  
  
        // leaf  
        if (root.left == null && root.right == null) {  
            return root.elem;  
        }  
  
        return sum_of_leaves(root.left) + sum_of_leaves(root.right);  
    }  
}
```

### Alternative

```
public static Integer sumOfLeaves( BSTNode root, Integer sum ){
    if(root==null){
        return 0;
    }
    if(root.left==null && root.right==null){
        return root.elem;
    }

    return sumOfLeaves(root.left, 0)+ sumOfLeaves(root.right, 0);
}
```

# Lab Assignment

## 4. Sum of range

Write a function **rangeSum(root,low,high)** that takes the root of a Binary Search Tree as a parameter and sums up the values within the range mentioned in the parameter.

| Sample Input                                                   | Sample Function Call  | Sample Returned Value               |
|----------------------------------------------------------------|-----------------------|-------------------------------------|
| <pre>      8      / \     4   12    / \ / \   2  6 10 14</pre> | rangeSum(root, 5, 13) | 36                                  |
|                                                                |                       | <b>Explanation:</b><br>6+8+10+12=36 |
| <pre>      10      / \     5   15    / \   \   3  7   18</pre> | rangeSum(root, 7, 15) | 32                                  |
|                                                                |                       | <b>Explanation:</b><br>7+10+15=32   |

```
public static Integer rangeSum( BSTNode root, Integer low, Integer high ){
    //Storing everything in an array
    Integer[] arr=new Integer[10];
    int n=1;

    fillArray(root, arr,n);

    //Finding sum after filling the array
    int sum=0;
    for(int i=1; i<arr.length; i++){
        if(arr[i]!=null && arr[i]>=low && arr[i]<=high){
            sum= sum+arr[i];
        }
    }

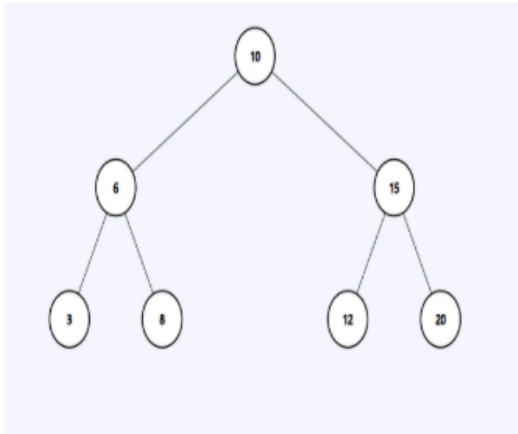
    return sum;
}
```

```
private static void fillArray(BSTNode root, Integer[] arr, int n){  
    if(root==null){  
        return;  
    }  
  
    arr[n]=root.elem;  
    fillArray(root.left, arr,2*n);  
    fillArray(root.right, arr, 2*n+1);  
  
    return;  
}
```

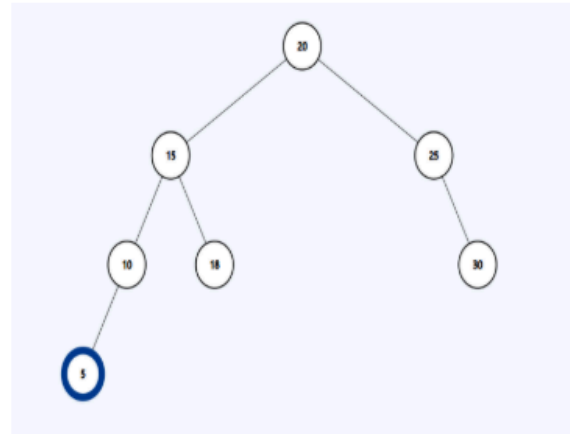
---



Example Tree input 1



Example Tree input 2



| Sample Input    | Sample Output | Explanation                                                                                                                                                        |
|-----------------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| mirrorSum(root) | 64            | For Tree 1 Mirror nodes are:<br>6 and 15, sum = 6 + 15 = 21<br>3 and 20, sum = 3 + 20 = 23<br>8 and 12, sum = 8 + 12 = 20<br>Total Mirror Node Sum = 21+23+20 = 64 |
| mirrorSum(root) | 80            | For Tree 2 Mirror nodes are:<br>15 and 25, sum = 15 + 25 = 40<br>10 and 30, sum = 10 + 30 = 40<br>Total Mirror Node Sum = 40 + 40 = 80                             |

```

public static Integer mirrorSum( BSTNode root ){
    if(root==null){
        return null;
    }

    int sum= addMirror(root.left, root.right); //Add main with mirror

    return sum;
}

private static int addMirror(BSTNode L, BSTNode R){
    if(R==null || L==null){
        return 0;
    }
  
```

```

return L.elem + R.elem + addMirror(L.left, R.right)+ addMirror(L.right, R.left);
}

```

---

## 6. Check BST

Write a function **isBST(root)** that takes the root of a Binary Tree as a parameter and then returns true if its a BST otherwise returns false.

| Sample Input                                               | Sample Returned Value |
|------------------------------------------------------------|-----------------------|
| <pre>       4      / \     9  2    / \  \   3 -5 7 </pre>  | false                 |
| <pre>       4      / \     2  7    / \  \  -5  3  9 </pre> | true                  |

```

public static Boolean isBST(BSTNode root ){
    if(root==null) return true;

    //For a specific node
    boolean leftAllSmaller= leftCheck(root.left, root);
    boolean rightAllBigger= rightCheck(root.right, root);

    if(leftAllSmaller==false || rightAllBigger==false){
        return false;
    }
}

```

```

//Baki child node r jonno
if(isBST(root.left)==false || isBST(root.right)==false){
    return false;
}
return true;
}

```

```

private static Boolean leftCheck(BSTNode root, BSTNode a){
    if(root==null) return true;

    int val=a.elem;
    if(root.elem>=val){
        return false;
    }

    if(leftCheck(root.left,a)==false || leftCheck(root.right,a)==false){
        return false;
    }
    return true;
}

```

```

private static Boolean rightCheck(BSTNode root, BSTNode a){
    if(root==null) return true;

    int val=a.elem;
    if(root.elem<=val){
        return false;
    }

    return rightCheck(root.left,a) && rightCheck(root.right,a);
}

```

# Lab Test (IMPORTANT)

1. Write a function **symmetricFactorDiff(root, x)** that takes the root of a BST and a node x.

- For the subtree rooted at x, consider all nodes with a symmetric sibling.
- For each symmetric pair, if one node is a factor of the other, add the absolute difference to the total sum.
- Ignore nodes without symmetric siblings.

**Note:** You must use recursion. You may use any number of helper functions as needed. You may assume that the `TreeNode` class has been defined for you.

| Input                                                              | Output                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>       8      / \     6  12    / \  \   3  7  24 x = 8 </pre> | <p><b>6 + 21 = 27</b></p> <p><b>Explanation:</b><br/> <math>6 \leftrightarrow 12</math>   <math>12 \% 6 == 0</math>   absolute difference: 6<br/> <math>3 \leftrightarrow 24</math>   <math>24 \% 3 == 0</math>   absolute difference: 21<br/> 7   no symmetric sibling   ignored</p> |

```

public static int symmetricFactorDiff(TreeNode root, int x) {
    TreeNode start = findNode(root, x);
    if (start == null) {
        return 0;
    }
    return compareMirror(start.left, start.right);
}

```

*//Find x*

```

private static TreeNode findNode(TreeNode root, int x) {
    if (root == null) {
        return null;
    }
    if ((int) root.elem == x) {
        return root;
    }
}

```

```

TreeNode left = findNode(root.left, x);
if (left != null) {
    return left;
}

```

```
}  
  
    return findNode(root.right, x);  
}
```

//Mirror add

```
private static int compareMirror(TreeNode a, TreeNode b) {  
    if (a == null || b == null) return 0;  
  
    int sum = 0;  
    int valA = (int) a.elem;  
    int valB = (int) b.elem;  
  
    if (valA % valB == 0 || valB % valA == 0) {  
        sum = Math.abs(valA - valB);  
    }  
  
    sum += compareMirror(a.left, b.right);  
    sum += compareMirror(a.right, b.left);  
  
    return sum;  
}
```

---

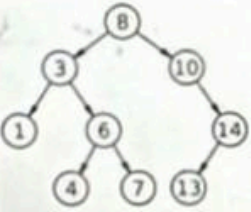
**Question:**

Given the root of a Binary Search Tree and an integer  $k$ , write a function `smallest_sum(root, k)` that returns the sum of the  $k$  smallest nodes of the BST.

You can assume  $k$  is always valid ( $0 \leq k \leq \text{total number of nodes}$ ).

**Constraints:**

- You must solve it using only recursion.
- No extra data structures like arrays, lists, or stacks.

| Example of BST                                                                    | Sample Inputs                      | Sample Outputs | Explanation                                  |
|-----------------------------------------------------------------------------------|------------------------------------|----------------|----------------------------------------------|
|  | <code>smallest_sum(root, 3)</code> | 8              | Sum 3 smallest nodes are: $1+3+4 = 8$        |
|                                                                                   | <code>smallest_sum(root, 5)</code> | 21             | Sum 5 smallest nodes are: $1+3+4+6+7 = 21$   |
|                                                                                   | <code>smallest_sum(root, 6)</code> | 29             | Sum 6 smallest nodes are: $1+3+4+6+7+8 = 29$ |

//BST r inorder korle inorder korle ascending hoy

```
class Solution {  
  
    int count = 0;  
    int sum = 0;  
  
    public int smallest_sum(TreeNode root, int k) {  
        inorder(root, k);  
        return sum;  
    }  
  
    private void inorder(TreeNode node, int k) {  
        if (node == null || count == k) return;  
  
        inorder(node.left, k);    //Ekdom end e jabe  
  
        if (count < k) {  
            sum += node.val;  
            count++;  
        }  
    }  
}
```

```

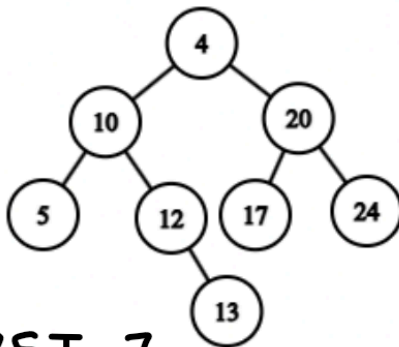
    }

    inorder(node.right, k);
}
}

```

---

## Find the number of BST



**BST-7**

//Counting

```

public static int countBST(BTNode root) {
    if (root == null) return 0;

    int count = 0;           //Alada count initialize cuz always add hobe na
    if (isBST(root) == true) {
        count = 1;
    }

    return count + countBST(root.left) + countBST(root.right);
}

```



//Check a specific node(From Top)

```
private static boolean isBST(BTNode root) {  
    if (root == null) return true;  
  
    boolean leftOK = allSmaller(root.left, root.elem);  
    boolean rightOK = allGreater(root.right, root.elem);  
  
    if (leftOK == false) return false;  
    if (rightOK == false) return false;  
  
    return isBST(root.left) == true &&  
        isBST(root.right) == true;  
}
```

//Left smaller check

```
private static boolean allSmaller(BTNode root, int val) {  
    if (root == null) return true;  
  
    if (root.elem >= val) return false;  
  
    return allSmaller(root.left, val) == true &&  
        allSmaller(root.right, val) == true;  
}
```

//Right bigger check

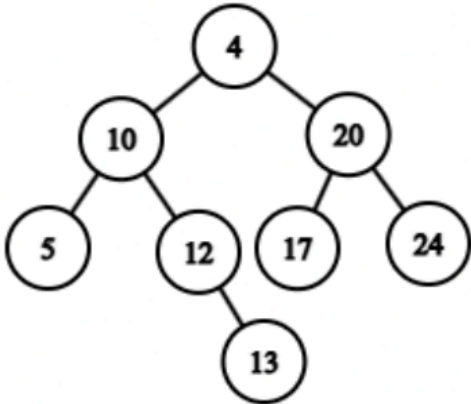
```
private static boolean allGreater(BTNode root, int val) {  
    if (root == null) return true;  
  
    if (root.elem <= val) return false;  
  
    return allGreater(root.left, val) == true &&  
        allGreater(root.right, val) == true;  
}
```

---

You are given a binary tree. Your task is to find the **size of the largest subtree that is a Binary Search Tree (BST)**.

**YOU CANNOT USE LIST OR DICTIONARY.** You cannot use any built-in function except for max function. You can create helper functions if needed.

[Hint: You may need three functions, one for checking if the tree is a BST and one for calculating the size of the tree and one for returning the largest BST]

| Sample Input                                                                       | Sample Output                                                                                                                                                                                      |
|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <b>4</b><br><b>Explanation:</b><br>The whole tree is not a BST<br>Left subtree is a BST with size 4<br>Right subtree is a BST with size 3<br>Hence, the largest subtree that is a BST is of size 4 |

(Similar to the previous one, just add one size method)

```
public static int largestBST(BTNode root) {  
    if (root == null) return 0;  
  
    if (isBST(root) == true) {  
        return size(root);  
    }  
  
    int leftMax = largestBST(root.left);
```

```
int rightMax = largestBST(root.right);
```

```
if (leftMax > rightMax) return leftMax;  
return rightMax;  
}
```

```
private static boolean isBST(BTNode root) {  
    if (root == null) return true;  
  
    boolean leftOK = allSmaller(root.left, root.elem);  
    boolean rightOK = allGreater(root.right, root.elem);  
  
    if (leftOK == false) return false;  
    if (rightOK == false) return false;  
  
    return isBST(root.left) == true &&  
        isBST(root.right) == true;  
}
```

```
private static boolean allSmaller(BTNode root, int val) {  
    if (root == null) return true;  
  
    if (root.elem >= val) return false;  
  
    return allSmaller(root.left, val) == true &&  
        allSmaller(root.right, val) == true;  
}
```

```
private static boolean allGreater(BTNode root, int val) {  
    if (root == null) return true;  
  
    if (root.elem <= val) return false;
```

```

return allGreater(root.left, val) == true &&
    allGreater(root.right, val) == true;
}

```

```

private static int size(BTNode root) {
    if (root == null) return 0;

    return 1 + size(root.left) + size(root.right);
}

```

| Sample Input                                                                                                                                                                                                                                                                                                                                                             | Sample Output                                                                                                                                                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>           20          / \         10  30        / \  \       5  15 40  def maxTreasure(root):     # return (path_str)     pass </pre> <hr/> <pre> static class Result {     int sum;     String path;     Result(int s, String p) {         sum = s;         path = p;     } }  public static Result maxTreasure(Node root) {     // write your code here } </pre> | <p><b>Output:</b> "20-&gt;30-&gt;40"</p> <p><b>Explanation:</b></p> <p>Paths are:</p> <p>20 → 10 → 5 = 35<br/> 20 → 10 → 15 = 45<br/> 20 → 30 → 40 = 90 (maximum)</p> <p>So the maximum treasure is <b>90</b>, and the best path is "20-&gt;30-&gt;40".</p> |

*Hint: The maximum path does not always lie in the right subtree.*

```

static class Result {

```

```

int sum;
String path;

Result(int s, String p) {
    sum = s;
    path = p;
}
}

```

```

public static Result maxTreasure(Node root) {

    // empty tree
    if (root == null)
        return new Result(0, "");

    // leaf node
    if (root.left == null && root.right == null)
        return new Result(root.elem, "" + root.elem);

    // choose the better side
    Result left = maxTreasure(root.left);
    Result right = maxTreasure(root.right);
    if (left.sum > right.sum) {
        return new Result(
            root.elem + left.sum,
            root.elem + "->" + left.path
        );
    }

    return new Result(
        root.elem + right.sum,
        root.elem + "->" + right.path
    );
}

```

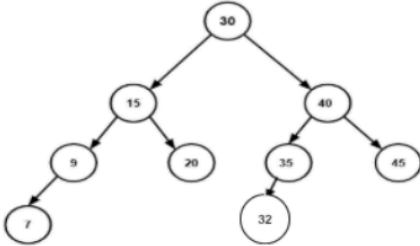
//Constructor call



# RFTS Practice

**Question:** Levy is playing an online game where each city is a node in a Binary Search Tree (BST), holding a visiting fare. Starting from the root, Levy must conquer cities along the path to his target. The total visiting fare is the sum of all node values on the route. If the target exists, the total fare is shown; otherwise, the system says "route does not exist."

Now, write a function **calculate\_fare(root, destination)** that takes the root of the BST and the target destination value as parameters and returns the total visiting fare or "route does not exist" if the destination is not found.

| Sample Input                                                                                                                                                                                                                                                                                                                    | Sample Output                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <p>calculate_fare(root,35)</p>  <pre> graph TD     30((30)) --&gt; 15((15))     30 --&gt; 40((40))     15 --&gt; 9((9))     15 --&gt; 20((20))     9 --&gt; 7((7))     40 --&gt; 35((35))     40 --&gt; 45((45))     35 --&gt; 32((32)) </pre> | <p>Total visiting fare = 105</p> <p>Explanation:</p> <p>Route : 30→40→35<br/>So, product = 30+40+35=105</p> |

```

public static int calculate_fare(BTNode root, int destination) {
    if (root.elem == destination) {
        return root.elem;
    }

    if (destination < root.elem) {
        return root.elem + calculate_fare(root.left, destination);
    }
    else {
        return root.elem + calculate_fare(root.right, destination);
    }
}

```